From Zero to Proximal Policy Optimization (PPO) in Reinforcement Learning (RL)

理论学习报告，参考书籍：《Mathematical Foundations of Reinforcement Learning》by Shiyu Zhao

Peng Ao

School of Control and Computer Engineering

2026 年 4 月 9 日

1. Fundamental Concepts
2. Model-based Algorithms
3. Model-free Algorithms
4. Function Approximation
5. Policy Gradient Methods
6. References

1 Fundamental Concepts

2 Model-based Algorithms

3 Model-free Algorithms

4 Function Approximation

5 Policy Gradient Methods

6 References

Fundamental Concepts in RL

- **State**: $s \in \mathcal{S}$ **Action**: $a \in \mathcal{A}$ **Reward**: $r \in \mathbb{R}$ **Policy**: $\pi(a|s)$
- **Episode/Trajectory**[1]: $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \cdots)$

$$S_t \xrightarrow{A_t} R_{t+1}, S_{t+1} \xrightarrow{A_{t+1}} R_{t+2}, S_{t+2} \xrightarrow{A_{t+2}} R_{t+3}, \cdots$$

- **Discounted Return**:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \quad \gamma \in [0, 1]$$

- **State Value Function**:

$$V_\pi(s) = \mathbb{E}\left[G_t \mid S_t = s\right] = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+1+k} \mid S_t = s\right]$$

- **Action Value Function**:

$$Q_\pi(s, a) = \mathbb{E}\left[G_t \mid S_t = s, A_t = a\right]$$

[1] Note that S_t, A_t, R_{t+1} are all **random variables**.

Derivation of Bellman Equation

$$
\begin{aligned}
\boldsymbol{V}_\pi(\boldsymbol{s}) &= \mathbb{E}\left[G_t \mid S_t = s\right] = \mathbb{E}\left[R_{t+1} + \gamma G_{t+1} \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} \mid S_t = s\right] + \gamma \mathbb{E}\left[G_{t+1} \mid S_t = s\right] \\
\mathbb{E}\left[R_{t+1} \mid S_t = s\right] &= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right] \\
&= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_r r \cdot p(r|s,a) \\
\mathbb{E}\left[G_{t+1} \mid S_t = s\right] &= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \mathbb{E}\left[G_{t+1} \mid S_t = s, A_t = a\right] \\
&= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{s'} p(s'|s,a) \mathbb{E}\left[G_{t+1} \mid S_{t+1} = s'\right] \\
&= \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{s'} p(s'|s,a) \boldsymbol{V}_\pi(\boldsymbol{s'})
\end{aligned}
$$

Bellman Equation

Bellman Equation:

$$\boldsymbol{V_\pi(s)} = \sum_a \pi(a|s) \left[\sum_r r \cdot p(r|s,a) + \gamma \sum_{s'} p(s'|s,a) \boldsymbol{V_\pi(s')} \right], \forall s \in \mathcal{S}$$

1. Bellman Equation describes the relationship among the value of all states.[2]
2. Given a policy π, we can obtain the value function $V_\pi(s)$ for each state $s \in \mathcal{S}$ by solving the Bellman Equation, which is called **policy evaluation**.
3. We can also obtain all action value function $Q_\pi(s,a)$ by the following equation:

$$\boldsymbol{Q_\pi(s,a)} = \sum_r r \cdot p(r|s,a) + \gamma \sum_{s'} p(s'|s,a) \boldsymbol{V_\pi(s')}, \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

[2] $p(r|s,a)$ and $p(s'|s,a)$ depend on the environment.

Optimal Policy

Optimal Policy:

$$\pi^* = \arg\max_{\pi} V_\pi(s), \quad \forall s \in \mathcal{S}$$

An optimal policy π^* is a policy that yields the highest value for all states compared to any other policy.

Greedy Policy:

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = a^*(s) \\ 0, & \text{otherwise} \end{cases}$$

where $a^*(s) = \arg\max_a Q_{\pi^*}(s, a)$ is the action that maximizes the action value function for state $s \in \mathcal{S}$ under the optimal policy π^*.

1 Fundamental Concepts

2 Model-based Algorithms
Policy Iteration Algorithm

3 Model-free Algorithms

4 Function Approximation

5 Policy Gradient Methods

6 References

Policy Iteration Algorithm

A simple idea is as follows: for a given initial policy π_0, we can obtain $V_\pi(s)$ for all states $s \in \mathcal{S}$ by solving the Bellman Equation(**policy evaluation**), and then derive $Q_\pi(s, a)$ for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$. Subsequently, a new policy π is obtained by the greedy strategy(**policy improvement**).

$$\pi(a|s) = \begin{cases} 1, & \text{if } a = \arg\max_a Q_\pi(s, a), \quad s \in \mathcal{S}, a \in \mathcal{A}(s) \\ 0, & \text{otherwise} \end{cases}$$

Repeatedly perform the above operations and continuously optimize the policy π until the optimal policy is reached, which is called the **Policy Iteration Algorithm**.

Policy Iteration Algorithm

Pseudocode: Policy iteration algorithm.

While the policy has not converged, for the kth iteration, do. (start from π_0)

Policy evaluation: $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$ (Solve Bellman Equation, use iterative solution)

$$\left(v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \; j = 0, 1, 2, \cdots \right)$$

Policy improvement: For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

$$q_{\pi_k}(s,a) = \sum_r p(r|s,a) \cdot r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')$$

$$a_k^*(s) = \arg\max_a q_{\pi_k}(s,a)$$

$\pi_{k+1}(a|s) = 1$ if $a = a^*$, and $\pi_{k+1}(a|s) = 0$ otherwise.

Policy Iteration Algorithm

Remarks:

- Note that we do not specifically discuss the **existence** of the optimal policy and the **convergence** of the algorithm. In fact, both are guaranteed.
- Meanwhile, this algorithm is **model-based**, because solving the Bellman Equation and computing $Q_\pi(s, a)$ from $V_\pi(s)$ both require the knowledge of $p(r|s, a)$ and $p(s'|s, a)$, which depend on the environment/model.
- This algorithm can only solve problems with finite State Space $\mathcal{S}$ and finite Action Space $\mathcal{A}$.

1 Fundamental Concepts

2 Model-based Algorithms

3 Model-free Algorithms
- Monte Carlo Method
- Temporal Difference (TD) Learning
- State-Action-Reward-State-Action (SARSA) Algorithm
- Q-learning Algorithm

4 Function Approximation

5 Policy Gradient Methods

6 References

Monte Carlo Method

- The idea of Monte Carlo method is to **estimate** $V_\pi(s)$ and $Q_\pi(s, a)$ by sampling trajectories τ from the environment under the policy π and calculating the discounted return G_t.

$$V_\pi(s) = \mathbb{E}\left[G_t \mid S_t = s\right] \approx \frac{1}{N_\tau(s)} \sum_{i=1}^{N_\tau(s)} g_t^{(i)}$$

  where $N_\tau(s)$ is the number of trajectories τ that pass through state s, and $g_t^{(i)}$ is the discounted return for the i-th trajectory at time t when it passes through state s.
- This is a **model-free** algorithm because it does not need to know $p(s'|s, a)$ and $p(r|s, a)$.
- **Disadvantage**: Because it requires multiple samples and needs to wait for the trajectory to end, this algorithm is not very efficient and is generally rarely used.

Temporal Difference (TD) Learning

- The idea of TD learning is to **estimate and update** $V_\pi(s)$ or $Q_\pi(s, a)$ by immediate rewards and existing predictions instead of full trajectories.

$$V_{t+1}(s_t) = V_t(s_t) - \alpha_t(s_t)\left\{V_t(s_t) - [r_{t+1} + \gamma V_t(s_{t+1})]\right\}$$

  where $V_t(s_t)$ is the **estimated state value** of the true value function $V_\pi(s_t)$, and $\alpha_t(s_t)$ is the learning rate.
- TD target: $r_{t+1} + \gamma V_t(s_{t+1})$
- TD error: $\delta_t = V_t(s_t) - [r_{t+1} + \gamma V_t(s_{t+1})]$

State-Action-Reward-State-Action (SARSA) Algorithm

- data/experience: $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$
- algorithm(update q-values and policy):

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)\left\{r_{t+1} + \gamma Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t)\right\}$$

$$Q_{t+1}(s, a) = Q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t)$$

where $Q_t(s_t, a_t)$ is the **estimated action value** of the true value function $Q_\pi(s_t, a_t)$, and $\alpha_t(s_t, a_t)$ is the learning rate.

$$\pi_{t+1}(a|s_t) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1), & a = \arg\max_a Q_{t+1}(s_t, a) \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{otherwise} \end{cases}$$

This is an ε-greedy policy, $\varepsilon \in [0, 1]$.

Q-learning Algorithm

- data/experience: $\{(s_t, a_t, r_{t+1}, s_{t+1})\}_t$ generated by a **behavior policy** π_b.
- algorithm:

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t \left\{ r_{t+1} + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right\},$$

$$Q_{t+1}(s, a) = Q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t)$$

Remarks:

- we can get the **optimal target policy** π_T^*:

$$\pi_T^*(a|s_t) = \begin{cases} 1, & a = \arg\max_a Q_{t+1}(s_t, a) \\ 0, & \text{otherwise} \end{cases}$$

  because $Q_t(s, a)$ will converge to $Q_{\pi^*}(s, a)$ as $t \to \infty$.
- we use greedy policy because this algorithm is an **off-policy** algorithm, which means that the policy used to generate data/experience is different from the policy being optimized.

1 Fundamental Concepts

2 Model-based Algorithms

3 Model-free Algorithms

4 Function Approximation
- Fundamental Concepts
- Optimization Algorithms
- Deep Q-Network (DQN) Algorithm

5 Policy Gradient Methods

6 References

Fundamental Concepts

- We want to find a function f that can approximate the true value function $V_\pi(s)$ or $Q_\pi(s,a)$, which is called **value function approximation**.
- **value function approximation** can solve the problem of large or continuous state and action space $\mathcal{S}, \mathcal{A}$ rather than the finite state and action space in the previous algorithms.
- The **objective function** $\mathcal{J}$ can be defined as the mean squared error between the true value function and the estimated value function:

$$\mathcal{J}(\omega) = \mathbb{E}\left[\left(V_\pi(S) - \hat{V}(S,\omega)\right)^2\right]$$

  where $\hat{V}(S,\omega)$ is the estimated value function parameterized by ω.
- Our goal is to find the best ω^* that minimizes $\mathcal{J}(\omega)$.
- we can use **Neural Networks** to approximate value function.

Function Approximation Optimization Algorithms

- We can use **Stochastic Gradient Descent (SGD)** to optimize the parameters ω of the function approximator:

$$\begin{aligned}\omega_{t+1} &= \omega_t - \alpha_t \nabla_\omega \mathcal{J}(\omega_t) \\ &= \omega_t + \alpha'_t \mathbb{E}\left[\left(V_\pi(S) - \hat{V}(S,\omega_t)\right)\nabla_\omega \hat{V}(S,\omega_t)\right] \\ &\approx \omega_t + \alpha'_t \left(V_\pi(s_t) - \hat{V}(s_t,\omega_t)\right)\nabla_\omega \hat{V}(s_t,\omega_t)\end{aligned}$$

- where $V_\pi(s_t)$ is the true value function, which is unknown. We can use the TD algorithm to replace $V_\pi(s_t)$ as follows:

$$\omega_{t+1} = \omega_t + \alpha_t \left(r_{t+1} + \gamma \widehat{V_{\omega_t}}(s_{t+1}) - \widehat{V_{\omega_t}}(s_t)\right)\nabla_\omega \widehat{V_{\omega_t}}(s_t)$$

- Sarsa with function approximation:

$$\omega_{t+1} = \omega_t + \alpha_t \left(r_{t+1} + \gamma \widehat{Q_{\omega_t}}(s_{t+1}, a_{t+1}) - \widehat{Q_{\omega_t}}(s_t, a_t)\right)\nabla_\omega \widehat{Q_{\omega_t}}(s_t, a_t)$$

DQN Algorithm

Q-learning with function approximation:

$$\omega_{t+1} = \omega_t + \alpha_t \left(r_{t+1} + \gamma \max_a \widehat{Q_{\omega_t}}(s_{t+1}, a) - \widehat{Q_{\omega_t}}(s_t, a_t) \right) \nabla_\omega \widehat{Q_{\omega_t}}(s_t, a_t)$$

- DQN is one of the earliest algorithms that introduce **deep neural networks** into RL.
- Note that DQN is essentially Q-learning with deep neural networks to **accelerate convergence and stabilize training**, optimizing the same objective function as follows:

$$\mathcal{J}(\omega) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \widehat{Q_\omega}(S', a) - \widehat{Q_\omega}(S, A) \right)^2 \right]$$

- we define $y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \widehat{Q_\omega}(S', a)$, we assume that ω in y is fixed, then we can calculate the gradient of $\mathcal{J}(\omega)$.

DQN Algorithm Techniques

Two Networks:

- To do that, we introduce two networks: the **main network** representing $\widehat{Q_\omega}(s,a)$ and the **target network** representing $\widehat{Q_{\omega_T}}(s,a)$, where ω_T is a copy of ω that is updated less frequently.
- when ω_T is fixed, we can calculate the gradient of $\mathcal{J}(\omega)$ as follows:

$$\nabla_\omega \mathcal{J}(\omega) = -2\mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \widehat{Q_{\omega_T}}(S', a) - \widehat{Q_\omega}(S, A)\right) \nabla_\omega \widehat{Q_\omega}(S, A)\right]$$

Experience Replay:

- replay buffer $\mathcal{B} \doteq \{(s, a, r, s')\}$, every time we train the network, we can draw a mini-batch of random samples from the $\mathcal{B}$ should follow a **uniform distribution**.
- because we assume the distribution of the (S, A) is **uniform distribution** in the expectation of $\mathcal{J}(\omega)$.

Remarks

- The above algorithms are called **value-based** algorithms because they focus on estimating the **value function**(Policy Evaluation), and based on this updating the policy (greedy or ε-greedy policy).
- We can also use **policy-based** algorithms(Policy Gradient Methods) to maximize the objective function $\mathcal{J}_\pi(\theta)$ to directly optimize the policy π without estimating the value function, which will be discussed in the next section.

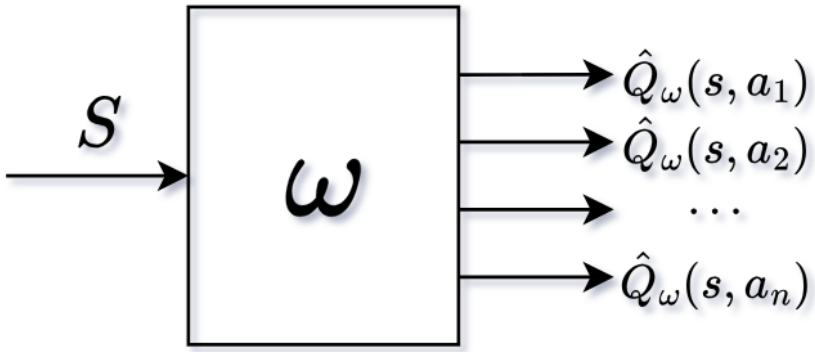

图 1: Action Value Neural Network Diagram.

1 Fundamental Concepts

2 Model-based Algorithms

3 Model-free Algorithms

4 Function Approximation

5 Policy Gradient Methods
- REINFORCE Algorithm
- Actor-Critic Algorithm
- Trust Region Policy Optimization (TRPO) Algorithm
- Proximal Policy Optimization (PPO) Algorithm

6 References

Fundamental Concepts

- We want to find a policy $\pi(a|s,\theta)$ that can directly optimize the **policy-based** objective function $\mathcal{J}(\theta)$, there are two metrics, the first one is the **average value**:

$$\bar{V}_\pi = \sum_{s\in\mathcal{S}} d(s) V_\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right]$$

- the second one is the **average reward**:

$$\bar{r}_\pi = \sum_{s\in\mathcal{S}} d(s) r_\pi(s) = \sum_{s\in\mathcal{S}} d(s) \sum_{a\in\mathcal{A}(s)} \pi(a|s) r(s,a) = (1-\gamma)\bar{V}_\pi$$

$$\bar{r}_\pi = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[R_{t+1} + R_{t+2} + \cdots + R_{t+n}\right] = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k}\right]$$

- Our goal is to find the best θ^* that maximizes the two metrics.
- we can use **Stochastic Gradient Ascent (SGA)** to optimize the policy $\pi(a|s,\theta)$: $\theta_{t+1} = \theta_t + \alpha_t \nabla_\theta \mathcal{J}(\theta_t)$

Gradient of Policy-based Objective Function

- We can derive $\nabla_\theta \mathcal{J}(\theta)$ by the **Policy Gradient Theorem**:

$$\nabla_\theta \mathcal{J}(\theta) = \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) Q_\pi(s,a)$$

  where $\mathcal{J}$ can be either $\bar{V}_\pi$ or $\bar{r}_\pi$.
- Therefore we can use SGA to optimize the policy $\pi(a|s,\theta)$:

$$\begin{aligned}
\theta_{t+1} &= \theta_t + \alpha_t \nabla_\theta \mathcal{J}(\theta_t) \\
&= \theta_t + \alpha_t \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) Q_\pi(s,a) \\
&= \theta_t + \alpha_t \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \pi(a|s,\theta) \nabla_\theta \ln \pi(a|s,\theta) Q_\pi(s,a) \\
&= \theta_t + \alpha_t \mathbb{E}_{S\sim\eta, A\sim\pi} \left[\nabla_\theta \ln \pi(A|S,\theta) Q_\pi(S,A)\right] \\
&\approx \theta_t + \alpha_t \nabla_\theta \ln \pi(a_t|s_t,\theta_t) Q_\pi(s_t,a_t)
\end{aligned}$$

REINFORCE Algorithm

- The SGA update rule can be rewritten as follows:

$$\theta_{t+1} = \theta_t + \alpha_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t) Q_\pi(s_t, a_t)$$

where $Q_\pi(s_t, a_t)$ is the true action value function, which is unknown. We can use $Q_t(s_t, a_t)$ to estimate $Q_\pi(s_t, a_t)$:

$$\theta_{t+1} = \theta_t + \alpha_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t) Q_t(s_t, a_t)$$

There are two ways to estimate $Q_t(s_t, a_t)$

- the first one is to use the **Monte Carlo Method** to estimate $Q_\pi(s_t, a_t)$ by sampling trajectories τ from the environment under the policy π and calculating the discounted return G_t, which is called the **REINFORCE Algorithm**.
- the second one is to use the **TD learning** to estimate $Q_\pi(s_t, a_t)$, which is called the **Actor-Critic Algorithm**.

The simplest Actor-Critic Algorithm(QAC)

- The idea of **Actor-Critic Algorithm** is to use a **critic** to estimate $Q_\pi(s, a)$, and use an **actor** to optimize the policy $\pi(a|s, \theta)$ by the policy gradient theorem(policy gradient + value function approximation).
- The update rule of the **actor** is as follows:

$$\theta_{t+1} = \theta_t + \alpha_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t) Q_t(s_t, a_t)$$

where $Q_t(s_t, a_t)$ is estimated by the **critic**.

- The update rule of the **critic** is as follows:

$$\omega_{t+1} = \omega_t + \beta_t \left(r_{t+1} + \gamma Q_{\omega_t}(s_{t+1}, a_{t+1}) - Q_{\omega_t}(s_t, a_t)\right) \nabla_\omega Q_{\omega_t}(s_t, a_t)$$

Note that the update rule of the **critic** is almost identical to the **Sarsa with function approximation**.

The principle of Advantage Actor-Critic Algorithm(A2C)

- The idea of **Advantage Actor-Critic Algorithm** is to use the **advantage function** $A_\pi(s,a) = Q_\pi(s,a) - V_\pi(s)$ to replace $Q_\pi(s,a)$ in the update rule of the **actor**, which can reduce the variance of the policy gradient estimate and improve the stability of training.
- Property: the policy gradient theorem still holds when we replace $Q_\pi(s,a)$ with $A_\pi(s,a)$, which means that the following equation still holds:

$$\begin{aligned}\nabla_\theta \mathcal{J}(\theta) &= \mathbb{E}_{S\sim\eta, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta) Q_\pi(S,A)\right] \\ &= \mathbb{E}_{S\sim\eta, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta)\left(Q_\pi(S,A) - V_\pi(S)\right)\right]\end{aligned}$$

- Furthermore, we can use the TD to estimate $A_\pi(s,a)$:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha_t \nabla_\theta \ln \pi(a_t|s_t,\theta_t) A_t(s_t,a_t) \\ &= \theta_t + \alpha_t \nabla_\theta \ln \pi(a_t|s_t,\theta_t)\left(r_{t+1} + \gamma V_{\omega_t}(s_{t+1}) - V_{\omega_t}(s_t)\right)\end{aligned}$$

Advantage Actor-Critic Algorithm(A2C)

- The update rule of the **critic** is as follows:

$$\omega_{t+1} = \omega_t + \beta_t \left(r_{t+1} + \gamma V_{\omega_t}(s_{t+1}) - V_{\omega_t}(s_t) \right) \nabla_\omega V_{\omega_t}(s_t)$$

- The update rule of the **actor** is as follows:

$$\theta_{t+1} = \theta_t + \alpha_t \left(r_{t+1} + \gamma V_{\omega_t}(s_{t+1}) - V_{\omega_t}(s_t) \right) \nabla_\theta \ln \pi(a_t | s_t, \theta_t)$$

Off-Policy A2C

- Initiallization: behavior policy π_b, target policy $\pi(a|s,\theta)$.
- **Importance Sampling Ratio**: $\frac{\pi(a_t|s_t,\theta)}{\pi_b(a_t|s_t)}$
- TD error:

$$\delta_t = r_{t+1} + \gamma V_{\omega_t}(s_{t+1}) - V_{\omega_t}(s_t)$$

- The update rule of the **critic** is as follows:

$$\omega_{t+1} = \omega_t + \beta_t \frac{\pi(a_t|s_t,\theta_t)}{\pi_b(a_t|s_t)} \delta_t \nabla_\omega V_{\omega_t}(s_t)$$

- The update rule of the **actor** is as follows:

$$\theta_{t+1} = \theta_t + \alpha_t \frac{\pi(a_t|s_t,\theta_t)}{\pi_b(a_t|s_t)} \delta_t \nabla_\theta \ln \pi(a_t|s_t,\theta_t)$$

Trust Region Policy Optimization (TRPO) Algorithm

- The core problem of early policy gradient methods lies in the sensitivity to the magnitude of policy updates. A single overly aggressive update can cause the agent to fall into a series of bad states that are difficult to recover from.
- TRPO introduces a **trust region constraint** to ensure that the updated policy does not deviate too much from the previous policy.

$$\mathcal{J}^{TRPO}(\theta) = \mathbb{E}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t\right]$$

subject to the constraint:

$$\mathbb{D}_{KL}\left(\pi_{\theta_{old}}(\cdot|s_t)||\pi_\theta(\cdot|s_t)\right) \leq \delta$$

where $\mathbb{D}_{KL}$ is the Kullback-Leibler divergence and δ is a hyperparameter that controls the size of the trust region.

Proximal Policy Optimization (PPO) Algorithm

- PPO introduces a **clipped surrogate objective function** to replace the original objective function $\mathcal{J}(\theta)$, which is defined as follows:

$$\mathcal{J}^{CLIP}(\theta) = \mathbb{E}\left[\min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}A_t, \text{clip}(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1-\varepsilon, 1+\varepsilon)A_t\right)\right]$$

where ε is a hyperparameter that controls the clipping range.

1 Fundamental Concepts

2 Model-based Algorithms

3 Model-free Algorithms

4 Function Approximation

5 Policy Gradient Methods

6 References

References

- S. Zhao. **Mathematical Foundations of Reinforcement Learning**. Springer, 2025.
- John Schulman. et al. **Trust region policy optimization**. In: CoRR, abs/1502.05477 (2015).
- John Schulman. et al. **Proximal policy optimization algorithms**. arXiv preprint arXiv:1707.06347, 2017.

Thanks

Thanks!

Peng Ao

2026 年 4 月 9 日